

WISDOM & CHANCE TCG

# Steam / Desktop Electron Client

---

## Implementation Plan for AI Agent Execution

Task #76 · Monorepo addition · No game logic changes required

6 Steps

~2-3 hrs agent

Windows · Mac ·  
Linux



This document is t

# Overview & Architecture

Wisdom & Chance TCG · Steam / Desktop Electron Client · Implementation Plan

## What This Task Builds

An /electron folder added to the existing monorepo that wraps the current React web frontend in an Electron desktop shell. The result is a native Windows, Mac, and Linux application that can be submitted to Steam. No server changes, no game logic changes, no shared/ changes.

## Monorepo Position After This Task

/client   !• React web frontend (unchanged)

/server   !• Shared Express backend (unchanged)

/shared   !• Shared types (unchanged)

/mobile   !• Expo mobile app (unchanged)

**/electron   !• NEW — Desktop/Steam client**

## How It Works

Electron is a framework that runs a web app inside a desktop window using a bundled Chromium browser. The /electron client loads the production server URL (<https://wisdom-and-chance.replit.app>) in that window — meaning the desktop app shares 100% of the same game code, UI, and backend as the web version. The only new code is the desktop shell itself and the login flow fix.

## Key Constraint — Login Flow

The Replit OAuth flow redirects the browser to <https://wisdom-and-chance.replit.app/api/callback> after the user logs in. In a normal browser this is transparent. In Electron, without intervention, it would open an external browser tab and leave the app stuck. The fix intercepts the navigation inside the Electron window using the will-navigate and did-navigate events so the callback completes inside the app. This is the only non-trivial technical challenge in the task.

## What Stays Out of Scope

- Steamworks SDK (achievements, cloud saves, Steam overlay) — separate future task
- Mac App Store or Microsoft Store — Steam only
- Offline mode — internet connection required, same as web
- Any changes to server, client, shared, or mobile directories

# Implementation Steps

Wisdom & Chance TCG · Steam / Desktop Electron Client · Implementation Plan

Execute steps in order. Each step is independent of the next except Step 2 which depends on Step 1 being scaffolded.

1

## Scaffold the Electron app

~25 min

Low

Create /electron/package.json with electron and electron-builder as dev dependencies.  
Create /electron/main.js — the Electron entry point. Open a BrowserWindow (1280×800 min) pointing at <https://wisdom-and-chance.replit.app>. Set the window title and icon using

2

## Fix the OAuth login flow for desktop

~40 min

Medium

In main.js, attach listeners to the BrowserWindow webContents for will-navigate and did-navigate events. When the URL contains /api/callback, allow it to resolve inside the window. This prevents the OAuth redirect from opening an external browser tab and

3

## Add dev workflow to .replit

~15 min

Low

Add a "start" script to /electron/package.json that runs: electron .  
Register a new "Start desktop app" workflow entry in .replit pointing at:  
cd electron && npm install && npm run start  
This lets the desktop app be launched independently from the workspace.

4

## Add production build script (electron-builder)

~35 min

Medium

Add a "build" script in /electron/package.json using electron-builder.  
Configure electron-builder with: appId (com.wisdomandchance.desktop), productName, icon path, and targets: Windows NSIS installer, Mac DMG, Linux AppImage.  
All three outputs should build from a single npm run build command.

5

## Document the Steam submission path

~10 min

Low

Create /electron/STEAM.md — a plain checklist of the four manual steps to get on Steam:  
(1) Pay Steam Direct \$100 fee, (2) Create a Steamworks app, (3) Upload Windows build via the Steamworks partner portal, (4) Set store page metadata. No code required.

6

## Update replit.md

~5 min

Low

Add the /electron client to the "Monorepo Structure" section of replit.md with a brief description matching the style of the existing /mobile entry. Note the workflow name, build command, and that it targets Steam/desktop distribution.

# Target File Structure & Key Code

Wisdom & Chance TCG · Steam / Desktop Electron Client · Implementation Plan

## New Files to Create

### `/electron/package.json`

Electron app manifest, scripts, electron-builder config

### `/electron/main.js`

Electron entry point — BrowserWindow + OAuth fix

### `/electron/STEAM.md`

Steam Direct submission checklist (manual steps)

## Files to Modify

### `.replit`

Add "Start desktop app" workflow entry

### `replit.md`

Add /electron to monorepo structure section

## Reference Files (read but do not modify)

- `client/public/icon-512.png` — app icon source
- `client/public/manifest.json` — app name / display name reference
- `script/build.ts` — existing esbuild config pattern to follow
- `package.json` (root) — existing script style to follow

## Key Code Patterns for `main.js`

---

*The agent should implement the following patterns. These are pseudocode outlines, not final code.*

```
// BrowserWindow setup
const win = new BrowserWindow({
  width: 1280, height: 800, minWidth: 1024, minHeight: 680,
  title: 'Wisdom & Chance',
  icon: path.join(__dirname, 'icon-512.png'),
  webPreferences: { nodeIntegration: false, contextIsolation: true }
});
win.loadURL('https://wisdom-and-chance.replit.app');

// OAuth callback interception
win.webContents.on('will-navigate', (event, url) => {
  // Allow all navigation within the app domain
  if (url.startsWith('https://wisdom-and-chance.replit.app')) return;
  // Block external navigations
  event.preventDefault();
  shell.openExternal(url);
});
```

# Time & Effort Estimate

Wisdom & Chance TCG · Steam / Desktop Electron Client · Implementation Plan

## Per-Step Breakdown

| Step                        | Estimated Agent Time | Difficulty |
|-----------------------------|----------------------|------------|
| 1 — Scaffold Electron app   | ~25 minutes          | Low        |
| 2 — Fix OAuth login flow    | ~40 minutes          | Medium     |
| 3 — Add dev workflow        | ~15 minutes          | Low        |
| 4 — Production build script | ~35 minutes          | Medium     |
| 5 — Steam.md checklist      | ~10 minutes          | Low        |
| 6 — Update replit.md        | ~5 minutes           | Low        |
| TOTAL                       | ~2 hrs 10 min        | —          |

### What "Agent Time" Means

AI agents execute in a single continuous session per task. For this task the agent will: read existing config files, install Electron packages, write all new files, modify .replit and replit.md, verify the server starts without errors, and confirm the desktop window opens. The 2–3 hour range accounts for package install time, any retry loops on the OAuth fix, and validation.

### Risk Areas That Could Add Time

- OAuth interception (Step 2)  
— if the Replit OIDC flow uses unusual redirect patterns, the agent may need a second pass to get it right. Add ~20 min buffer.

- electron-builder target configuration (Step 4) — cross-platform build targets have version-specific quirks. Add ~15 min buffer if issues arise.

- Icon format — electron-builder requires .icns on Mac and .ico on Windows in addition to .png. The agent will need to handle format conversion or use a PNG fallback.

## Prerequisites Before Starting

- The published app (wisdom-and-chance.replit.app) must be live and reachable — the desktop app loads it directly.

- No new secrets or API keys are needed.

- No database changes required.

- Agent must have internet access to install npm packages (standard in Replit).

## Recommended Validation After Completion

- Run "Start desktop app" workflow — a native window should appear showing the game

- Click "Sign in with Replit" — login should complete inside the window, no external tab

- Navigate to deck builder, shop, and multiplayer lobby — all should load normally

- Run npm run build from /electron — should produce .exe, .dmg, and .AppImage files

# Steam Submission Path (Manual Steps Post-Build)

Wisdom & Chance TCG · Steam / Desktop Electron Client · Implementation Plan

---

*These steps are performed by the developer manually after the Electron build is complete. They are NOT automated by the agent.*

## Step 1 — Pay the Steam Direct Fee

Go to [partner.steamgames.com](https://partner.steamgames.com) and pay the \$100 USD Steam Direct fee per game. This is a one-time fee that is recouped once the game earns \$1,000 in revenue.

## Step 2 — Create a Steamworks App

In the Steamworks partner portal, create a new application. You will receive an App ID. Update the `/electron/main.js` with this App ID if you integrate Steamworks SDK later.

## Step 3 — Upload the Windows Build

Use the Steamworks partner portal build uploader (SteamPipe) to upload the Windows .exe installer produced by electron-builder. Mac and Linux builds can be added as separate depots for the same App ID.

## Step 4 — Set Store Page Metadata

Fill in the Steam store page: game description, screenshots (use the same ones from your Kickstarter), trailer video link, tags (Card Game, Strategy, Multiplayer), pricing, and system requirements. Use the content from the Kickstarter Story section as your starting point.

## Step 5 — Submit for Review

Steam reviews new games before they go live. Review typically takes 3–5 business days. Once approved, set your release date and publish.

## Future: Steamworks SDK Integration (separate task)

Once the base Electron client is live, a follow-up task can add the Steamworks SDK via the greenworks Node.js bindings. This enables:

- Steam Achievements (map to the existing in-game achievement system)
- Steam Cloud Saves (sync deck data)
- Steam Overlay (in-game browser and friend notifications)
- Steam Friends integration (invite Steam friends to multiplayer games)

---

*Steamworks integration is out of scope for Task #76. Plan it as a separate task after the base client is working and on Steam.*